Control inalámbrico por Bluetooth con ESP32 e interacción con pantalla OLED

Oct 30, 2026 · Electrónica Digital II · Universidad Surcolombiana · Grupo 4

Código	Nombre completo
20241220023	Sebastián Serrato Ramírez
20241220316	David Santiago Mondragón
20241220762	Ximena del Rocío Moreno

Docente: Ing. Johan Julián Molina Mosquera

1. Objetivo

Diseñar e implementar un sistema que recibe comandos desde un teléfono por Bluetooth Low Energy (BLE) y los ejecuta en una pantalla OLED I2C conectada a un ESP32: escribir texto libre, borrar la pantalla y elegir la posición y el tamaño de la letra.

2. Concebir: comunicación y comandos

El ESP32 integra Wi-Fi y Bluetooth 4.2 (Classic y BLE). Como el teléfono del grupo es un iPhone, que no admite el perfil serie clásico SPP, se usó **BLE con el servicio UART de Nordic (NUS)**. Este servicio emula un puerto serie con dos características: **RX**, donde el teléfono escribe los comandos, y **TX**, por donde el ESP32 responde mediante notificaciones. El ESP32 actúa como servidor y se anuncia con el nombre `ESP32_Grupo4`.

Comando	Ejemplo	Acción en la OLED	Respuesta al teléfono
Texto libre	`Hola USCO`	Escribe el texto en la posición actual y baja una línea	`OK texto: Hola USCO`
`TXT:` + mensaje	`TXT:Aviso`	Igual que el texto libre	`OK texto: Aviso`

Comando	Ejemplo	Acción en la OLED	Respuesta al teléfono
`BORRAR`	`BORRAR`	Borra la pantalla y vuelve a (0,0)	`OK pantalla borrada`
`POS:x,y`	`POS:0,32`	Mueve el cursor a x, y en píxeles (x 0–127, y 0–63)	`OK posicion 0,32`
`TAM:n`	`TAM:2`	Tamaño de letra 1, 2 o 3 (8, 16 o 24 px de alto)	`OK tamano 2`
`AYUDA`	`AYUDA`	—	Lista de comandos

3. Diseñar

3.1 Circuito

Pin OLED (SSD1306 128×64)	Pin ESP32	Justificación
VCC	3V3	El módulo trabaja a 3,3 V, el mismo nivel lógico del ESP32
GND	GND	Referencia común
SDA	GPIO21	Línea de datos I2C por defecto del ESP32
SCL	GPIO22	Línea de reloj I2C por defecto del ESP32

El módulo OLED ya trae las resistencias pull-up del bus I2C, así que no se agregan componentes externos. La dirección I2C de la pantalla es 0x3C. El ESP32 se alimenta por USB, que también sirve para programarlo.

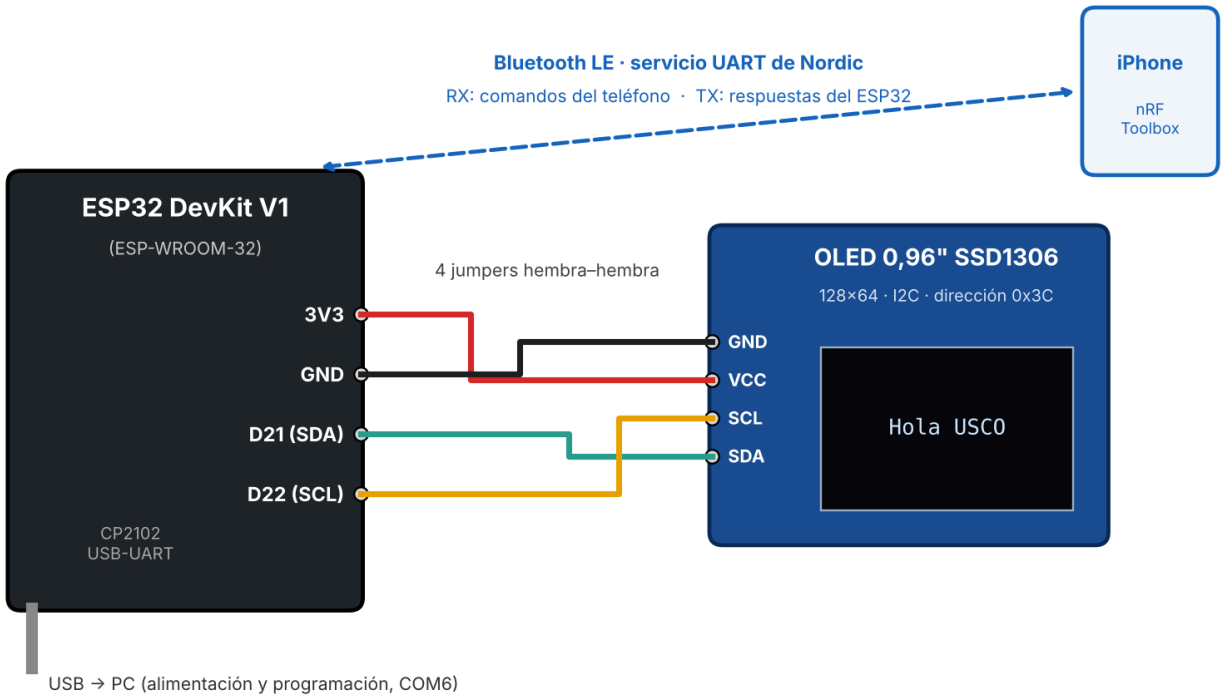


Figura 1. Diagrama de conexiones: la OLED va al bus I2C del ESP32 (GPIO21/22) y el iPhone se comunica por BLE.

3.2 Interfaz GUI en el teléfono

Se usó la aplicación **nRF Toolbox** (Nordic Semiconductor) en un iPhone. Al tocar *Connect to Device* y elegir `ESP32_Grupo4`, la app detecta el **UART Service** y abre una pantalla tipo chat: un campo de texto con el botón *Send* para enviar cada comando y un historial donde aparecen en azul los comandos enviados y en verde las respuestas del ESP32 (por ejemplo `OK pantalla borrada`). Esta pantalla funciona como la interfaz gráfica del sistema: no fue necesario programar una app propia.

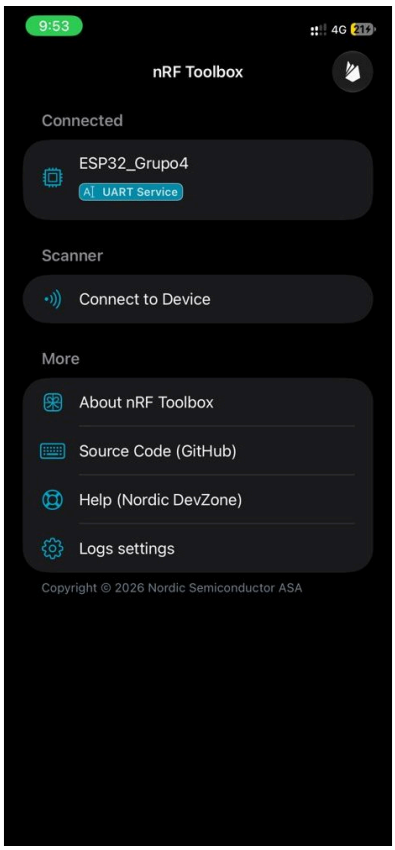


Figura 2. nRF Toolbox conectado a ESP32_Grupo4: la app reconoce el servicio UART.

3.3 Lógica del programa

1. `setup()`: inicia el I2C y la OLED, muestra un mensaje de espera, crea el servidor BLE con el servicio UART y empieza a anunciarse.
2. Cuando el teléfono escribe en RX, el evento `onWrite` solo guarda el comando y levanta una bandera. Así no se usa el I2C dentro del evento de Bluetooth.
3. `loop()` revisa la bandera y llama a `procesar()`, que identifica el comando (BORRAR, TAM:, POS:, AYUDA, TXT: o texto libre), actúa sobre la OLED y responde por TX.
4. Si el teléfono se desconecta, el ESP32 vuelve a anunciarse para aceptar otra conexión.

Librerías: `Adafruit_SSD1306` y `Adafruit_GFX` para la pantalla, y `BLEDevice`, `BLEServer` y `BLE2902` del paquete ESP32 de Espressif para Bluetooth.

4. Implementar

El programa se escribió en lenguaje Arduino (C++) y se compiló con el paquete **esp32 de Espressif v3.3.12** para la placa *ESP32 Dev Module*, usando el `arduino-cli` incluido en Arduino IDE. Para que Windows reconociera la placa se instaló el driver **Silicon Labs**

CP210x del convertidor USB-UART CP2102; después de eso la placa apareció como **COM6** y el programa se cargó por USB. La OLED se conectó con cuatro jumpers hembra–hembra directamente a los pines del ESP32, sin protoboard.

```ino
/*
  Laboratorio: Control inalámbrico por Bluetooth con ESP32 - Grupo 4 (pantalla OLED I2C)
  Electrónica Digital II - Universidad Surcolombiana

  El celular (iPhone, app nRF Toolbox -> UART) envía comandos por Bluetooth BLE
  y el ESP32 los muestra o los ejecuta en una pantalla OLED SSD1306 de 128x64.

  Conexiones OLED -> ESP32:  VCC -> 3V3 | GND -> GND | SDA -> GPIO21 | SCL -> GPIO22

  Comandos (se escriben en el celular):
    cualquier texto  -> lo escribe en la pantalla (texto libre)
    TXT:mensaje      -> igual, escribe "mensaje"
    BORRAR           -> borra la pantalla y vuelve a la esquina superior
    POS:x,y          -> mueve el cursor a la posición x,y en píxeles (x 0-127, y 0-63)
    TAM:n            -> tamaño de letra n = 1, 2 o 3
    AYUDA            -> devuelve la lista de comandos al celular
*/

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

// ---------- Pantalla OLED ----------
#define ANCHO 128
#define ALTO  64
Adafruit_SSD1306 oled(ANCHO, ALTO, &Wire, -1);

// ---------- BLE: servicio UART de Nordic (lo reconocen nRF Toolbox y Bluefruit) ----------
#define UUID_SERVICIO "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define UUID_RX       "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"   // celular -> ESP32
#define UUID_TX       "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"   // ESP32 -> celular

BLECharacteristic *txChar;
bool conectado = false;
String comando = "";
volatile bool hayComando = false;

int posX = 0, posY = 0, tam = 1;   // estado del cursor y tamaño de letra
bool hayOled = false;              // false si la pantalla no responde (permite probar el BLE sin ella)

// Envía una respuesta al celular y al Monitor Serie
void responder(const String &msg) {
  Serial.println(msg);
  if (conectado) {
    txChar->setValue(msg.c_str());
    txChar->notify();
  }
}

// Eventos de conexión
class EventosServidor : public BLEServerCallbacks {
  void onConnect(BLEServer *s) override {
    conectado = true;
  }
  void onDisconnect(BLEServer *s) override {
    conectado = false;
    s->getAdvertising()->start();       // vuelve a ser visible para reconectar
  }
};

// Llega un comando desde el celular: solo se guarda; se procesa en loop()
class EventosRx : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *c) override {
    comando = String(c->getValue().c_str());
    comando.trim();
    hayComando = true;
  }
};

// Escribe texto en la posición y tamaño actuales
void escribir(const String &texto) {
  if (hayOled) {
    oled.setTextSize(tam);
    oled.setCursor(posX, posY);
```

```
    oled.print(texto);
    oled.display();
  }
  posY += 8 * tam;                      // baja a la siguiente línea
  if (posY >= ALTO) posY = 0;
  responder("OK texto: " + texto);
}

// Interpreta cada comando
void procesar(const String &cmd) {
  String may = cmd;
  may.toUpperCase();

  if (may == "BORRAR") {
    if (hayOled) {
      oled.clearDisplay();
      oled.display();
    }
    posX = 0;
    posY = 0;
    responder("OK pantalla borrada");
  }
  else if (may.startsWith("TAM:")) {
    int t = cmd.substring(4).toInt();
    if (t >= 1 && t <= 3) {
      tam = t;
      responder("OK tamano " + String(tam));
    } else {
      responder("ERROR: usa TAM:1, TAM:2 o TAM:3");
    }
  }
  else if (may.startsWith("POS:")) {
    int coma = cmd.indexOf(',');
    if (coma > 4) {
      posX = constrain(cmd.substring(4, coma).toInt(), 0, ANCHO - 1);
      posY = constrain(cmd.substring(coma + 1).toInt(), 0, ALTO - 1);
      responder("OK posicion " + String(posX) + "," + String(posY));
    } else {
      responder("ERROR: usa POS:x,y  ej. POS:0,32");
    }
  }
  else if (may == "AYUDA") {
    responder("Comandos: texto | TXT:msg | BORRAR | POS:x,y | TAM:1-3");
  }
  else if (may.startsWith("TXT:")) {
    escribir(cmd.substring(4));
  }
  else if (cmd.length() > 0) {
    escribir(cmd);                      // texto libre
  }
}
```

Figura 3. Código (parte 1): librerías, servicio BLE UART, eventos y procesamiento de comandos.

```
void setup() {
  Serial.begin(115200);

  // OLED por I2C en los pines por defecto del ESP32
  Wire.begin(21, 22);
  hayOled = oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  if (!hayOled) {
    Serial.println("No se encontro la OLED (revisa cables y direccion 0x3C). El Bluetooth sigue funcionando.");
  } else {
    oled.clearDisplay();
    oled.setTextColor(SSD1306_WHITE, SSD1306_BLACK);   // con fondo: sobrescribe limpio
    oled.setTextSize(1);
    oled.setCursor(0, 0);
    oled.println("ESP32 + OLED + BLE");
    oled.println("Grupo 4");
    oled.println("Esperando conexion...");
    oled.display();
  }
  posY = 32;

  // Bluetooth BLE con el servicio UART
  BLEDevice::init("ESP32_Grupo4");
  BLEServer *servidor = BLEDevice::createServer();
  servidor->setCallbacks(new EventosServidor());
  BLEService *servicio = servidor->createService(UUID_SERVICIO);

  txChar = servicio->createCharacteristic(UUID_TX, BLECharacteristic::PROPERTY_NOTIFY);
  txChar->addDescriptor(new BLE2902());

  BLECharacteristic *rxChar = servicio->createCharacteristic(
      UUID_RX, BLECharacteristic::PROPERTY_WRITE | BLECharacteristic::PROPERTY_WRITE_NR);
  rxChar->setCallbacks(new EventosRx());

  servicio->start();
  BLEAdvertising *anuncio = BLEDevice::getAdvertising();
  anuncio->addServiceUUID(UUID_SERVICIO);
  anuncio->setScanResponse(true);
  BLEDevice::startAdvertising();
  Serial.println("Listo: busca 'ESP32_Grupo4' en el celular");
}

void loop() {
  if (hayComando) {
    hayComando = false;
    procesar(comando);
  }
  delay(10);
}
```

Figura 4. Código (parte 2): setup() con la inicialización de la OLED y del servidor BLE, y loop().

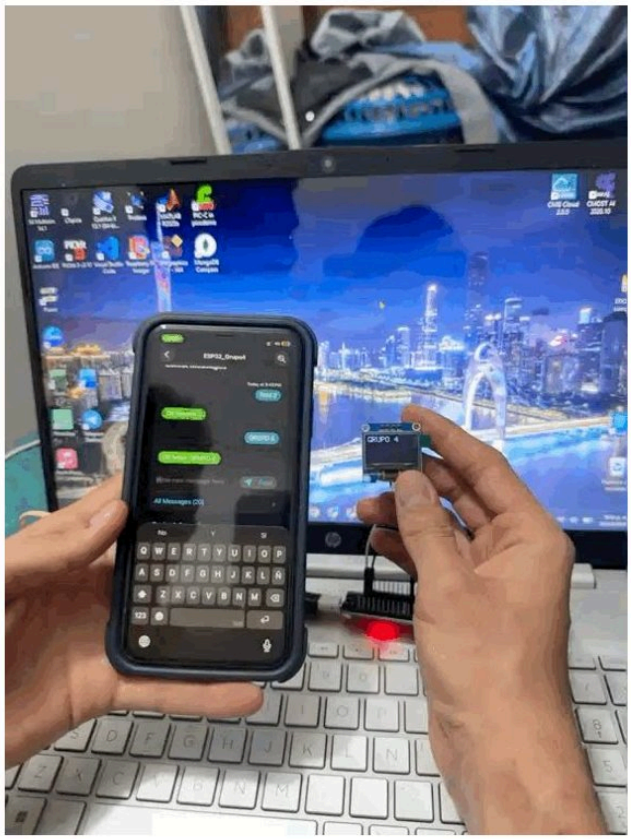


Figura 5

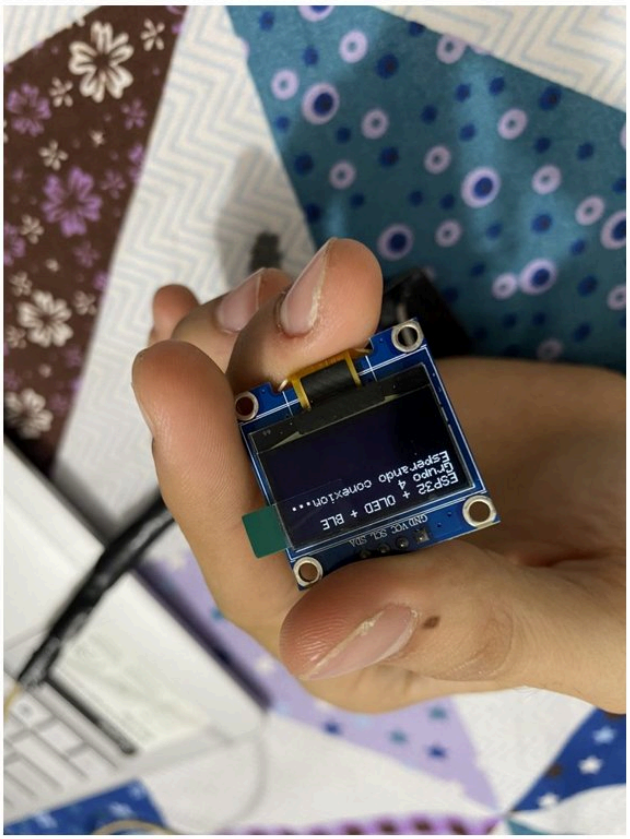


Figura 6

Figura 5. Montaje: ESP32 alimentado por USB, OLED conectada por I2C y el iPhone enviando comandos. Figura 6. Mensaje de inicio después de cargar el programa: "ESP32 + OLED + BLE / Grupo 4 / Esperando conexion...".

5. Operar: pruebas

Prueba	Comando enviado	Resultado esperado	Resultado obtenido
Conexión	—	El iPhone encuentra y se conecta a ESP32_Grupo4	Cumple: conecta y reconoce el UART Service (Fig. 2)
Texto libre	`Hola`, `Holaaaa`, `Jeje jeje` …	Cada texto aparece en una línea nueva	Cumple. Al llegar al final de la pantalla el cursor vuelve arriba y escribe sobre lo anterior (Fig. 7)

Prueba	Comando enviado	Resultado esperado	Resultado obtenido
Borrar	`BORRAR`	Pantalla vacía y respuesta `OK pantalla borrada`	Cumple (Fig. 8)
Tamaño y posición	`TAM:2` → `GRUPO 4`; `TAM:1` → `POS:0,30` → `Electronica digital II`; `POS:0,50` → `USCO 2026`	Tres textos en tamaños y alturas distintas	Cumple. La última «I» pasó a la línea siguiente (Fig. 9 y 10)
Tamaño 3 en posición	`BORRAR` → `TAM:3` → `POS:20,20` → `HOLA`	Texto grande cerca del centro	Cumple (Fig. 11)
Ayuda	`AYUDA`	Lista de comandos en el teléfono	Cumple: responde `Comandos: texto | TXT:msg | BORRAR | POS:x,y | TAM:1-3` (Fig. 12)
Comando inválido	TAM:9	Mensaje de error y la OLED no cambia	Cumple: responde «ERROR: usa TAM:1, TAM:2 o TAM:3» (Fig. 13)

Análisis de resultados

Todos los comandos funcionaron y cada uno recibió su confirmación en el teléfono, lo que demuestra la comunicación en los dos sentidos (RX y TX). Se observaron dos comportamientos que se explican por el diseño:

- **Salto de línea automático.** Con `TAM:1` cada carácter mide 6 × 8 px, así que en 128 px caben 128 / 6 ≈ 21 caracteres. «Electronica digital II» tiene 22, por eso la última «I» bajó a la línea siguiente. Con `TAM:2` caben unos 10 caracteres y con `TAM:3` unos 7.
- **Sobrescritura al llegar al final.** Después de cada texto el cursor baja 8 × tamaño píxeles; si pasa de 63 vuelve a y = 0 y escribe sobre lo que había arriba (Fig. 7). Para evitarlo se usa `BORRAR` o se elige la posición con `POS`. Una mejora posible es desplazar el texto hacia arriba (scroll) o borrar automáticamente al llegar abajo.

La latencia entre enviar un comando y verlo en la pantalla fue imperceptible a simple vista (estimación cualitativa, no medida).

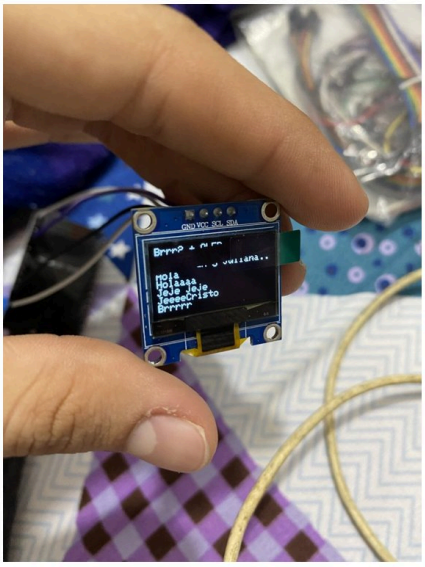


Figura 7

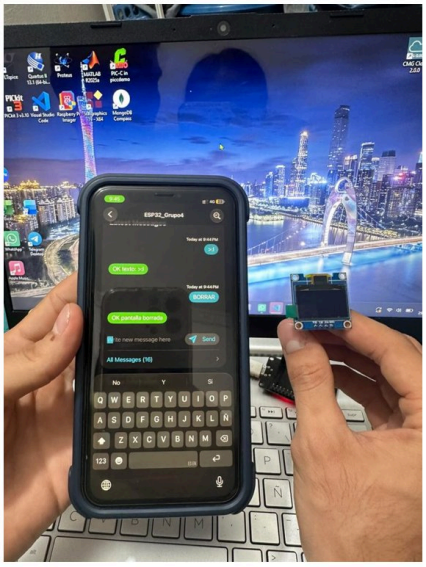


Figura 8

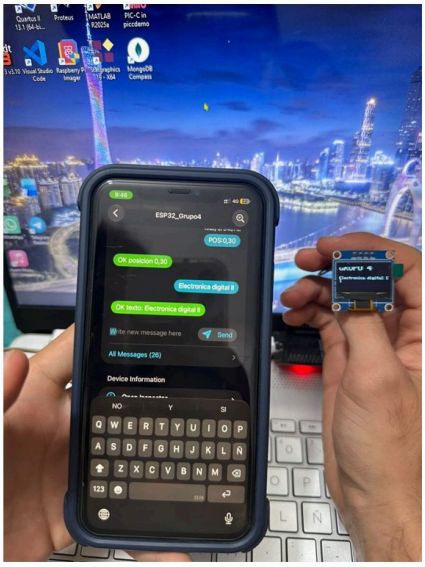


Figura 9

Figura 7. Texto libre: cada mensaje en una línea; al llegar abajo el texto vuelve a empezar arriba y sobrescribe el mensaje inicial. Figura 8. BORRAR: pantalla vacía y respuesta «OK pantalla borrada». Figura 9. «GRUPO 4» con TAM:2 y «Electronica digital II» con TAM:1 en POS:0,30.

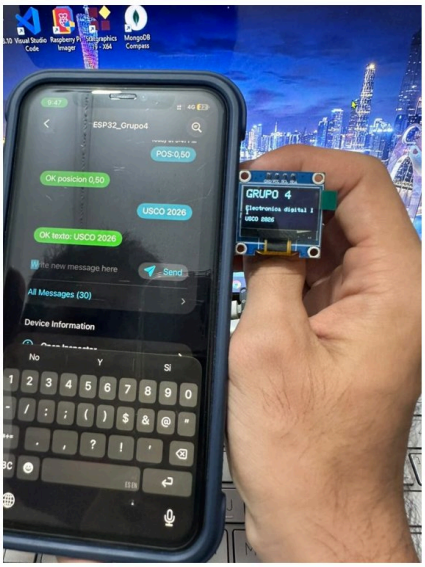


Figura 10

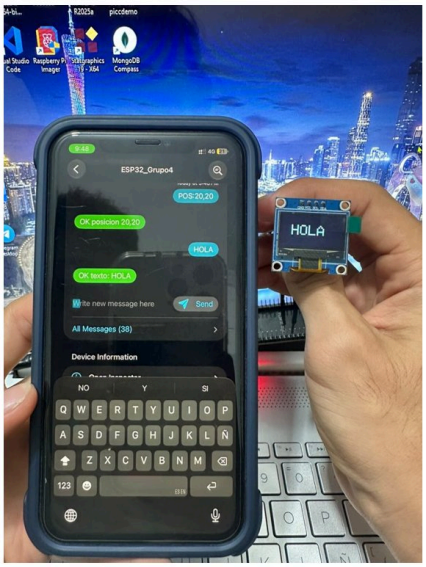


Figura 11

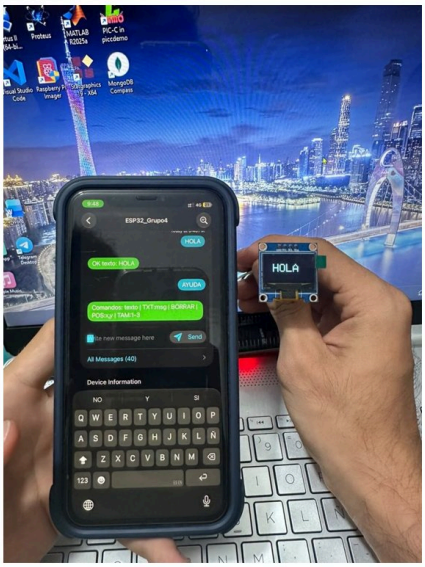


Figura 12

Figura 10. Tres textos con distinto tamaño y posición; «USCO 2026» en POS:0,50. Figura 11. «HOLA» con TAM:3 en POS:20,20. Figura 12. Respuesta del ESP32 al comando AYUDA en nRF Toolbox.



Figura 13. Validación de comandos: al enviar TAM:9 el ESP32 responde con un mensaje de error y no modifica la pantalla.

6. Conclusiones

- BLE con el servicio UART permite enviar comandos de texto desde cualquier teléfono, incluido iPhone, sin red Wi-Fi ni hardware adicional.
- Un protocolo de comandos simple (palabra clave y parámetros) hace que el sistema sea fácil de ampliar y que los errores de usuario se detecten y se respondan.
- Procesar los comandos en `loop()` y no dentro del evento BLE evita conflictos entre el Bluetooth y el bus I2C.
- Para programar la placa en Windows fue necesario un cable USB **de datos** (no solo de carga) y el driver del convertidor CP2102; sin el driver el dispositivo aparece con advertencia y no se crea el puerto COM.
- El tamaño de letra limita cuántos caracteres caben por línea (≈21, 10 y 7 para TAM 1, 2 y 3), así que los mensajes deben planearse según el espacio de la pantalla de 128 × 64 px.

Código completo

El código completo aparece en las Figuras 3 y 4 y se entrega también en el archivo `ESP32_OLED_BLE.ino`.

Referencias

[1] Espressif Systems, *ESP32 Series Datasheet*, 2023. [En línea]. Disponible: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Solomon Systech, *SSD1306 128×64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, 2008.

[3] Nordic Semiconductor, *Nordic UART Service (NUS)*. [En línea]. Disponible: https://docs.nordicsemi.com

[4] Adafruit Industries, *Adafruit SSD1306 Library*. [En línea]. Disponible: https://github.com/adafruit/Adafruit_SSD1306

[5] Silicon Labs, *CP210x USB to UART Bridge VCP Drivers*. [En línea]. Disponible: https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers